

Reviewer Pack — Minimal Rank of Quantum Entanglement and Communication Compl...

Entry 4f4b77d18cf7 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-11 22:34:12 UTC

Minimal Rank of Quantum Entanglement and Communication Complexity

Entry ID: 4f4b77d18cf7 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-11 22:34:12 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory - Entanglement Theory **Field B** (complexity object): Communication Complexity - Rank Variance

Statement:

The minimal rank of the quantum entanglement tensor for a given Boolean function φ is linearly correlated with its communication complexity rank variance, such that $\min_rank(\Psi_\varphi) = \Theta(\text{rank_var}(\varphi))$.

Rationale (proposer's reasoning):

Entanglement theory provides a mathematical framework to understand complex quantum systems. If there is a quantitative relationship between the entanglement rank and the communication complexity rank, it might reveal deep connections between quantum phenomena and classical computational tasks.

Taxonomy category: QuantumEntanglementRankVsCommunicationComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 68590ed5c16bdc2a

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

For the conjecture that the minimal rank of the quantum entanglement tensor $\min_rank(\Psi_\varphi)$ is linearly correlated with communication complexity rank variance $\text{rank_var}(\varphi)$, support will be indicated if the correlation coefficient from a linear regression model on $n \leq 40$ instances, using 30 seeds, exceeds 0.8 and the p-value is less than 0.01.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank quantum entanglement" AND "communication complexity rank variance" - "entanglement tensor Boolean function" AND communication complexity - "quantum information theory entanglement theory" AND communication complexity rank variance

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=1.4s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_boolean_function(n):
        return [random.choice([0, 1]) for _ in range(2**n)]

    def compute_quantum_state(phi):
        n = len(phi)
        state = [[0] * (2**n) for _ in range(2**n)]
        for i in range(2**n):
            for j in range(2**n):
                if phi[i ^ j] == 1:
                    state[i][j] = 1
        return state

    def min_rank(state):
        n = len(state)
        rank = 0
        for i in range(n):
            row = [state[j][i] for j in range(n)]
            if any(row):
                rank += 1
        return rank

    def communication_complexity(phi):
        n = len(phi)
        max_comm = 0
```

```

    for i in range(2**n):
        for j in range(i+1, 2**n):
            if phi[i] != phi[j]:
                comm = bin(i ^ j).count('1')
                if comm > max_comm:
                    max_comm = comm
    return max_comm

def rank_variance(phi):
    n = len(phi)
    comm = communication_complexity(phi)
    return (comm - 2)**2

n_values = [5, 10, 15, 20, 30, 40]
min_ranks = []
rank_vars = []

for n in n_values:
    phi = generate_boolean_function(n)
    state = compute_quantum_state(phi)
    min_rank_value = min_rank(state)
    rank_var_value = rank_variance(phi)

    min_ranks.append(min_rank_value)
    rank_vars.append(rank_var_value)

if not min_ranks or not rank_vars:
    return {
        "metric_name": "min_rank vs rank_var",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

n = len(min_ranks)
mean_min_rank = sum(min_ranks) / n
mean_rank_var = sum(rank_vars) / n

def linear_regression(x, y):
    n = len(x)
    x_mean = sum(x) / n
    y_mean = sum(y) / n

    numerator = sum((x[i] - x_mean) * (y[i] - y_mean) for i in range(n))
    denominator = sum((x[i] - x_mean)**2 for i in range(n))

    if denominator == 0:
        return None, None

    slope = numerator / denominator
    intercept = y_mean - slope * x_mean

    return slope, intercept

```

```

slope, _ = linear_regression(min_ranks, rank_vars)

if slope is None:
    return {
        "metric_name": "min_rank vs rank_var",
        "metric_value": None,
        "instances_tested": n,
        "n_max": max(n_values),
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

    return {
        "metric_name": "min_rank vs rank_var",
        "metric_value": slope,
        "instances_tested": n,
        "n_max": max(n_values),
        "conjecture_holds": abs(slope) >= 0.8,
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(x) for x in sys.argv[1:]] or [2**i + 7 for i in range(5, 30)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if not all(results):
        return

    mean_slope = sum(r["metric_value"] for r in results) / len(results)
    support_fraction = sum(1 for r in results if abs(r["metric_value"]) >= 0.8) / len(results)

    if support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_slope} std=NA support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not abs(result["metric_value"]) >= 0.8)
        print(f"RESULT: FALSIFIED counterexample=\"slope too low\" first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_1b241283.py", line 133
    return
    ^^^^^
SyntaxError: 'return' outside function

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a syntax error, preventing any data from being produced or analyzed. | next: Review the test code for syntax errors and correct them before running the test again.

11. Audit log (LLM calls)

Total LLM calls: 11

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	11479
2	propose	ollama_remote	glm4:latest	0	0	12732
3	propose	ollama_remote	glm4:latest	0	0	12844
4	preregistration	ollama_remote	glm4:latest	0	0	9656
5	novelty	ollama_remote	glm4:latest	0	0	8389
6	novelty	ollama_remote	glm4:latest	0	0	10262
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	44734
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13612
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11268
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15012
11	judge	ollama_remote	glm4:latest	0	0	53736

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 203723 ms total latency. Provider mix: {'ollama_remote': 11}

(full prompt+response transcripts available in research/audit/4f4b77d18cf7.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/4f4b77d18cf7.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/4f4b77d18cf7.tar.gz (if generated)